

Teaching Type Inference

Making auto tangible in Selfie

Florian Schroffner

June 25, 2026

University of Salzburg

Everyone uses it. Nobody sees how.

```
auto var :=
```

- In C++, Rust, Go
- The type just appears
- Taught as deep theory
- Students never see it work

How do you *teach* it?

Shrink it until you can read it

`auto p = malloc(8);` $\longrightarrow$ `p : pointer`

- Selfie: a tiny compiler
- Small enough to read
- `auto`: type from initialiser

But where does the type come from?

The whole grammar change

A statement can now also be an auto declaration:

```
statement  =  ... | call ";" | return ";"  
            | auto_declaration ";" .  
  
auto_declaration  =  "auto" identifier "=" expression .
```

- One new production
- `expression` is reused
- Initialiser required; `void` rejected

The type is already there

```
auto p = malloc(8);
```

```
inferred_type = compile_expression();
```

- Every expression returns its type
- auto just keeps it
- No new analysis

One line does the inference.

And that is the whole feature

Part	Language	Size
auto implementation	C* (selfie.c)	~55 lines, 3 functions
Test programs	C*	14 files, 136 lines
Autograder suite	Python	24 automated checks

- One new function
- No new pass, no AST
- Compiles *and* rejection tested

Small surface, fully tested.

Small enough to understand

- A real feature, fully built
- Readable end to end
- The mechanism made visible

Type inference is not magic —
it is the type the compiler
already had.